SQL Complete Guide — From Fresher to Expert

Every topic below has a **1-line definition** and an **example query**. Follow the sections top to bottom — each one builds on the previous.

---

0. How to Write a SQL Query, and How It Executes Internally

How to write a query — the standard structure:

```
SELECT column1, column2       -- what to display
FROM table_name               -- which table
WHERE condition               -- filter rows
GROUP BY column               -- group rows
HAVING group_condition        -- filter groups
ORDER BY column               -- sort result
LIMIT n;                      -- restrict number of rows
```

The order you *write* a query is not the order it *executes* in. This is the single most important thing to understand early.

Writing order (top to bottom): SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

Actual execution order (internally):

1. FROM → identify the table(s), perform JOINs
2. WHERE → filter individual rows
3. GROUP BY → group the remaining rows
4. HAVING → filter the groups
5. SELECT → pick/compute the columns to return
6. ORDER BY → sort the final result
7. LIMIT → cut down to the requested number of rows

Why this matters: you can't use a column alias created in SELECT inside a WHERE clause, because WHERE runs *before* SELECT — but you *can* use it in ORDER BY, since that runs *after* SELECT.

What happens inside the database engine when you run a query:

1. **Parser** — checks SQL syntax and converts the query into a structure the engine understands.
2. **Query Optimizer** — looks at available indexes, table sizes, and statistics to choose the fastest **execution plan**.
3. **Execution Engine** — carries out the plan: fetching rows (via index or full scan), applying filters, joining tables, grouping, and sorting.
4. **Result** — the final rows are returned to you.

You can inspect this plan yourself using EXPLAIN:

EXPLAIN SELECT * FROM student WHERE age > 18;

This shows whether the database will scan the whole table or use an index — the starting point for tuning slow queries later on.

---

1. SQL Command Categories

SQL commands are grouped into five categories based on what they do.

DDL (Data Definition Language) — defines or changes the structure of database objects.

CREATE TABLE student (id INT PRIMARY KEY, name VARCHAR(50));

DML (Data Manipulation Language) — manipulates the actual data inside tables.

INSERT INTO student (id, name) VALUES (1, 'Varun');

DQL (Data Query Language) — retrieves data (sometimes classified under DML).

SELECT * FROM student;

DCL (Data Control Language) — controls access and permissions.

GRANT SELECT ON student TO 'analyst_user';

TCL (Transaction Control Language) — manages transactions.

COMMIT;

---

2. Data Types

INT / INTEGER — whole numbers, no decimals.

age INT

VARCHAR(n) — variable-length text up to n characters.

name VARCHAR(100)

CHAR(n) — fixed-length text, padded with spaces if shorter.

gender CHAR(1)

DECIMAL(p,s) / NUMERIC(p,s) — exact decimal numbers, useful for money.

price DECIMAL(10,2)

DATE / DATETIME / TIMESTAMP — store date and/or time values.

created_at TIMESTAMP

BOOLEAN — stores TRUE or FALSE.

is_active BOOLEAN

TEXT — large blocks of text with no strict length limit.

description TEXT

---

3. Creating and Modifying Tables

CREATE TABLE — creates a new table with defined columns.

```
CREATE TABLE student (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  age INT
);
```

ALTER TABLE (ADD COLUMN) — adds a new column to an existing table.

ALTER TABLE student ADD COLUMN email VARCHAR(100);

ALTER TABLE (MODIFY/ALTER COLUMN) — changes a column's data type or definition.

ALTER TABLE student MODIFY COLUMN age SMALLINT;

ALTER TABLE (DROP COLUMN) — removes a column from a table.

ALTER TABLE student DROP COLUMN email;

RENAME TABLE — changes a table's name.

ALTER TABLE student RENAME TO students;

DROP TABLE — permanently deletes a table and all its data.

DROP TABLE students;

TRUNCATE TABLE — quickly removes all rows but keeps the table structure.

TRUNCATE TABLE students;

Difference — DELETE vs TRUNCATE vs DROP: DELETE removes rows (can filter, can rollback); TRUNCATE removes all rows fast (can't filter, resets identity); DROP removes the entire table structure and data permanently.

---

4. Keys and Constraints

PRIMARY KEY — uniquely identifies each row; cannot be NULL or duplicate.

id INT PRIMARY KEY

FOREIGN KEY — links a column to the primary key of another table, enforcing valid references.

FOREIGN KEY (customer_id) REFERENCES customer(id)

UNIQUE — ensures all values in a column are different (allows one NULL).

email VARCHAR(100) UNIQUE

NOT NULL — ensures a column can never be left empty.

name VARCHAR(100) NOT NULL

CHECK — restricts values in a column based on a condition.

age INT CHECK (age >= 18)

DEFAULT — sets an automatic value when none is provided.

status VARCHAR(20) DEFAULT 'active'

COMPOSITE KEY — a primary key made of two or more columns together.

PRIMARY KEY (student_id, course_id)

ON DELETE CASCADE — automatically deletes child rows when the parent row is deleted.

FOREIGN KEY (customer_id) REFERENCES customer(id) ON DELETE CASCADE

---

5. Inserting, Updating, and Deleting Data

INSERT INTO — adds new rows into a table.

INSERT INTO student (id, name, age) VALUES (1, 'Varun', 20);

INSERT multiple rows — adds several rows in a single statement.

INSERT INTO student (id, name, age) VALUES (2, 'Ajay', 22), (3, 'Sandhya', 21);

UPDATE — modifies existing rows that match a condition.

UPDATE student SET age = 21 WHERE id = 1;

DELETE — removes rows that match a condition.

DELETE FROM student WHERE id = 3;

---

6. Basic Querying (SELECT)

SELECT specific columns — retrieves only the chosen columns.

SELECT name, age FROM student;

SELECT * — retrieves all columns from a table.

SELECT * FROM student;

WHERE — filters rows based on a condition.

SELECT * FROM student WHERE age > 18;

DISTINCT — removes duplicate values from the result.

SELECT DISTINCT age FROM student;

ORDER BY — sorts the result set (ASC by default, or DESC).

SELECT * FROM student ORDER BY age DESC;

LIMIT — restricts how many rows are returned.

SELECT * FROM student LIMIT 5;

AS (Alias) — gives a column or table a temporary display name.

SELECT name AS student_name FROM student;

---

7. Operators

Comparison operators (=, !=, >, <, >=, <=) — compare values directly.

SELECT * FROM student WHERE age >= 18;

AND / OR / NOT — combine multiple conditions.

SELECT * FROM student WHERE age > 18 AND name = 'Varun';

BETWEEN — checks if a value falls within a range (inclusive).

SELECT * FROM student WHERE age BETWEEN 18 AND 25;

IN — checks if a value matches any value in a given list.

SELECT * FROM student WHERE age IN (18, 20, 22);

LIKE — matches text using wildcard patterns (% = any characters, _ = one character).

SELECT * FROM student WHERE name LIKE 'V%';

IS NULL / IS NOT NULL — checks whether a column has no value.

SELECT * FROM student WHERE email IS NULL;

---

8. Aggregate Functions

COUNT() — counts the number of rows.

SELECT COUNT(*) FROM student;

SUM() — adds up numeric values.

SELECT SUM(price) FROM product;

AVG() — calculates the average of numeric values.

SELECT AVG(age) FROM student;

MAX() / MIN() — finds the largest / smallest value.

SELECT MAX(price), MIN(price) FROM product;

GROUP BY — groups rows sharing the same value in a column, usually paired with an aggregate function.

SELECT course_id, COUNT(*) FROM student_course GROUP BY course_id;

HAVING — filters groups *after* aggregation (unlike WHERE, which filters rows before).

```
SELECT course_id, COUNT(*) AS total
FROM student_course
GROUP BY course_id
HAVING COUNT(*) > 5;
```

---

9. JOINs

INNER JOIN — returns only the rows that match in both tables.

```
SELECT student.name, course.name
FROM student
INNER JOIN student_course ON student.id = student_course.student_id
INNER JOIN course ON student_course.course_id = course.id;
```

LEFT JOIN — returns all rows from the left table, plus matches from the right (NULL if no match).

```
SELECT student.name, student_course.course_id
FROM student
LEFT JOIN student_course ON student.id = student_course.student_id;
```

RIGHT JOIN — returns all rows from the right table, plus matches from the left.

```
SELECT course.name, instructor.full_name
FROM course
RIGHT JOIN instructor ON course.instructor_id = instructor.instructor_id;
```

FULL JOIN — returns all rows from both tables, matched where possible.

```
SELECT course.name, instructor.full_name
FROM course
FULL JOIN instructor ON course.instructor_id = instructor.instructor_id;
```

CROSS JOIN — returns every possible combination of rows from both tables.

```
SELECT student.name, course.name FROM student CROSS JOIN course;
```

SELF JOIN — a table joined with itself, used to compare rows within the same table.

```
SELECT a.student_id, b.student_id, a.course_id
FROM student_course a
INNER JOIN student_course b
  ON a.course_id = b.course_id AND a.student_id < b.student_id;
```

NATURAL JOIN — automatically joins tables using columns that share the same name.

```
SELECT * FROM course NATURAL JOIN instructor;
```

---

10. Subqueries

Subquery in WHERE — a query nested inside another query's condition.

```
SELECT name FROM student
WHERE id IN (SELECT student_id FROM student_course WHERE course_id = 11);
```

Subquery in SELECT — computes a value for each row using a nested query.

```
SELECT name, (SELECT COUNT(*) FROM student_course sc WHERE sc.student_id =
student.id) AS total_courses
FROM student;
```

Correlated Subquery — a subquery that references a column from the outer query, so it re-runs for every row.

```
SELECT name FROM student s
WHERE EXISTS (SELECT 1 FROM student_course sc WHERE sc.student_id = s.id);
```

EXISTS — checks whether a subquery returns any rows at all.

```
SELECT name FROM student s
WHERE EXISTS (SELECT 1 FROM review r WHERE r.student_id = s.id);
```

---

11. Set Operators

UNION — combines results of two queries and removes duplicate rows.

```
SELECT name FROM student
UNION
SELECT full_name FROM instructor;
```

UNION ALL — combines results of two queries but keeps duplicates (faster than UNION).

```
SELECT name FROM student
UNION ALL
SELECT full_name FROM instructor;
```

INTERSECT — returns only the rows that appear in both queries' results.

```
SELECT student_id FROM student_course WHERE course_id = 11
INTERSECT
SELECT student_id FROM review WHERE course_id = 11;
```

MINUS / EXCEPT — returns rows from the first query that do not appear in the second (MINUS in Oracle, EXCEPT in SQL Server/PostgreSQL).

```
SELECT student_id FROM student
EXCEPT
SELECT student_id FROM student_course;
```

---

12. Views

View — a saved SQL query that acts like a virtual table, without storing data itself.

```
CREATE VIEW active_students AS
SELECT * FROM student WHERE age >= 18;
```

Querying a View — used exactly like a regular table.

```
SELECT * FROM active_students;
```

Dropping a View — removes the saved view definition.

```
DROP VIEW active_students;
```

---

13. Indexes

Index — a data structure that speeds up data retrieval at the cost of extra storage and slower writes.

```
CREATE INDEX idx_student_name ON student(name);
```

Unique Index — an index that also enforces uniqueness of values.

```
CREATE UNIQUE INDEX idx_student_email ON student(email);
```

Dropping an Index — removes an index that's no longer needed.

```
DROP INDEX idx_student_name;
```

---

14. Stored Procedures, Functions & Triggers

Stored Procedure — a saved, reusable block of SQL logic that can be executed as a single call.

```
CREATE PROCEDURE GetStudentsByAge(IN min_age INT)
BEGIN
  SELECT * FROM student WHERE age >= min_age;
END;
```

Calling it:

```
CALL GetStudentsByAge(18);
```

Function — similar to a procedure, but always returns a single value and can be used inside a query.

```
CREATE FUNCTION GetStudentAge(sid INT) RETURNS INT
BEGIN
  RETURN (SELECT age FROM student WHERE id = sid);
END;
```

Trigger — a block of SQL that runs automatically when a specific event (INSERT/UPDATE/DELETE) happens on a table.

```
CREATE TRIGGER before_student_insert
BEFORE INSERT ON student
FOR EACH ROW
SET NEW.name = UPPER(NEW.name);
```

---

15. Transactions & ACID

Transaction — a group of SQL statements executed as a single logical unit — all succeed, or none do.

```
BEGIN TRANSACTION;
UPDATE account SET balance = balance - 500 WHERE id = 1;
UPDATE account SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

ROLLBACK — undoes all changes made in the current transaction.

```
ROLLBACK;
```

SAVEPOINT — marks a point within a transaction to roll back to, without undoing everything.

```
SAVEPOINT before_update;
```

ACID Properties (one-line each):

- **Atomicity** — the transaction is all-or-nothing.
- **Consistency** — the database moves from one valid state to another.
- **Isolation** — concurrent transactions don't interfere with each other.
- **Durability** — committed changes survive even a crash.

---

16. Normalization (Designing Clean Tables)

Normalization — organizing tables to reduce redundancy and avoid data inconsistency.

1NF — every column holds a single, atomic value (no repeating groups).

```
-- Bad: phone_numbers = "9999999999, 8888888888"
-- Good: separate row per phone number in its own table
```

2NF — must be in 1NF, and every non-key column must depend on the *whole* primary key.

```
-- If (student_id, course_id) is the key, "student_name" shouldn't be in this table —
-- it only depends on student_id, not the full key.
```

3NF — must be in 2NF, and no non-key column should depend on another non-key column.

```
-- "instructor_name" shouldn't sit in the course table if it depends on instructor_id,
-- not directly on course_id.
```

---

17. ER Model to Relational Database (Practical Flow)

Entity → Table — every real-world object becomes a table.

```
CREATE TABLE customer (id INT PRIMARY KEY, name VARCHAR(100));
```

Attribute → Column — every property of the entity becomes a column.

```
CREATE TABLE product (id INT PRIMARY KEY, name VARCHAR(100), price INT);
```

One-to-Many (1:M) — the foreign key goes on the "many" side.

```
CREATE TABLE address (
  id INT PRIMARY KEY,
  city VARCHAR(100),
  customer_id INT,
  FOREIGN KEY (customer_id) REFERENCES customer(id)
);
```

One-to-One (1:1) — the foreign key gets a UNIQUE constraint.

```
CREATE TABLE cart (
  id INT PRIMARY KEY,
  customer_id INT UNIQUE,
  FOREIGN KEY (customer_id) REFERENCES customer(id)
);
```

Many-to-Many (M:M) — a separate junction table holds both foreign keys.

```
CREATE TABLE cart_product (
  cart_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (cart_id) REFERENCES cart(id),
  FOREIGN KEY (product_id) REFERENCES product(id)
);
```

---

18. Advanced SQL — Window Functions & CTEs

Window Function — performs a calculation across a set of rows related to the current row, without collapsing them like GROUP BY does.

```
SELECT name, score,
```

```
  RANK() OVER (ORDER BY score DESC) AS rank_position
FROM student_course;
```

ROW_NUMBER() — assigns a unique sequential number to each row within a partition.

```
SELECT name, ROW_NUMBER() OVER (PARTITION BY course_id ORDER BY score
DESC) AS position
FROM student_course;
```

CTE (Common Table Expression / WITH clause) — a temporary, named result set used to simplify complex queries.

```
WITH high_scorers AS (
  SELECT student_id FROM student_course WHERE score > 80
)
SELECT * FROM student WHERE id IN (SELECT student_id FROM high_scorers);
```

Recursive CTE — a CTE that refers to itself, often used for hierarchical data (like org charts).

```
WITH RECURSIVE subordinates AS (
  SELECT id, manager_id FROM employee WHERE id = 1
  UNION ALL
  SELECT e.id, e.manager_id FROM employee e
  INNER JOIN subordinates s ON e.manager_id = s.id
)
SELECT * FROM subordinates;
```

---

19. Query Optimization & Execution Plans

EXPLAIN — shows how the database plans to execute a query (index usage, join order, scan type).

```
EXPLAIN SELECT * FROM student WHERE age > 18;
```

Full Table Scan — the engine reads every row because no suitable index exists — slow on large tables.

Index Scan — the engine uses an index to jump directly to matching rows — much faster.

Rule of thumb — add indexes on columns frequently used in WHERE, JOIN, and ORDER BY, but avoid over-indexing since every index slows down INSERT/UPDATE/DELETE.

---

20. SQL Injection & Security Basics

SQL Injection — an attack where malicious SQL is inserted through user input to manipulate a query.

-- Dangerous (string concatenation):
"SELECT * FROM student WHERE name = '" + userInput + "'"

Prevention — Parameterized Queries — user input is passed as a parameter, never concatenated directly into SQL.

SELECT * FROM student WHERE name = ?; -- placeholder bound safely by the driver

---